



DB Traveler (Uni Hildesheim Edition)

Advanced Web Development (AWD) Final Project

Dear Dr. Holger Eichelberger and Dr. Klaus Schmidt,

Welcome to our Advanced Web Development (AWD) final project submission. This `README.md` provides a comprehensive overview of the **DB Traveler** application, explaining precisely how to run it, what it does, and how it fulfills all the Technical (T), Mandatory (MR), Standard (SR), and Custom (CR) requirements outlined for this course.

Team Members

Name	Matriculation Number
Poalelungi, Ion	1751886
Chiriciuc, Anna	1751897
Kasten, Erik Otto	1751656

How to Run the Application

To make reviewing as easy as possible, we have set up an automated script on the homescreen of the provided VirtualBox VM. VM password - 1111. A simple pin. Link to the VM-

<https://drive.google.com/drive/folders/1dlhX6h6jW1EGFVI7gO8kl6qz4LeOwpTt?usp=sharing>

Automated Execution via Homescreen Script:

1. Double-click the launch script located directly on the **Desktop/Homescreen**.
2. The script will automatically:
 - **Start the FastAPI Backend** on `http://127.0.0.1:8000`.
 - **Start the Angular Frontend** on `http://localhost:4200`.
 - **Run the Test Suite** in a separate terminal window. You will see the Karma test runner execute the tests in a headless Chrome environment, calculate the coverage (line, branch, function >60%), and display the coverage report in the terminal alongside code linting results.

Sample Test Data: Included in the root directory, you will find several **ready-to-use, downsized .csv files**. These files have intentionally been made smaller so that testing the upload functionalities, long-running calculations, and gamification rewards takes a reasonable amount of time without blocking the application for too long. The files are located on desktop in folder: DBtrainrides_part_14.

(Note: If you wish to run the app manually, `cd backend && uvicorn main:app --reload`, then in a new terminal `cd awd-frontend && npm start`, and for tests `ng test --code-coverage`.)

Application Overview & Architecture

The application allows users to upload, view, filter, sort, and analyze German train data. It operates on a decoupled architecture:

- **Frontend:** Built with Angular 17+ using Standalone Components, Signals for state management, RxJS for reactive data streams, and Angular Material for UI dialogs and progress bars.
 - **Backend:** Built with Python FastAPI, providing a fully RESTful API, asynchronous background tasks (for CSV uploads and complex delays/cancellation odds), and JWT-based authentication.
-

REST API Documentation & Communication

The Python FastAPI backend provides a fully documented REST API that serves as the backbone for the Angular frontend.

It automatically generates an interactive interface using Swagger UI. Once the application is running via the script, you can explore, test, and view the precise schemas for every endpoint by navigating to: <http://127.0.0.1:8000/docs>

API Modules Overview

Our API is logically grouped into the following domains to support the decoupled architecture:

- **Authentication & Users** (`/api/auth` , `/api/users`): Handles user registration, login (issuing JWTs), logout, profile management, password updates, and retrieving gamification points for the leaderboard.
- **Train Data Management** (`/api/trains`): Facilitates data retrieval, including paginated and filtered train lists, detailed lookups, CSV downloads, and the asynchronous uploading of new CSV train data.
- **Analytics & Compute Tasks** (`/api/analytics`): Endpoints that trigger our complex, longer-running calculations (like cancellation odds and average delays). It also provides polling endpoints (`/api/analytics/status/{task_id}`) that the frontend uses to track progress in real-time.
- **Routing** (`/api/routing`): Supplies coordinate data for stations and hierarchal Bundesland (state) categorizations.

How the Context Works (Frontend ↔ Backend)

1. When a user logs in, the backend's `/api/auth/login` endpoint validates the credentials and returns a secure **JSON Web Token (JWT)**.
 2. The Angular frontend stores this JWT locally and uses an `HttpInterceptor` to automatically attach it as a `Bearer` token to the `Authorization` header of every subsequent outgoing HTTP request.
 3. If an endpoint requires heavy computation (like uploading a large CSV), the backend immediately responds with a `task_id`. The frontend then utilizes RxJS observables to continuously poll the API with that `task_id`, updating the UI's progress bar smoothly until the backend signals the task is complete.
-

Detailed Requirements Fulfillment Mapping

Below is the mapping of your grading criteria to our codebase for better requirement readability. For brevity, not all requirements are stated here, please consult the final requirements file. Thank you.

Technical Requirements

- **T01 & T03:** The frontend is exclusively built using **Angular**, communicating via HTTP standard requests (JWT Bearer Auth) with a decoupled **Python FastAPI backend**.
- **T02:** The system utilizes the **Angular Router** to provide a persistent, main navigation menu, allowing users to switch between Dashboard, Data Table, Uploads, Analytics, and their User Profile without full page reloads.
- **T04 & T05:** Test coverage strictly uses **Karma** (`karma.conf.js` configured with `karma-coverage` and `ChromeHeadless`). The terminal output proves the app achieves **>60%** line, branch, and function coverage. E2E workflows have also been incorporated.
- **T06 & T08:** This entire project is packaged and submitted as a runnable **VirtualBox VM**, complete with documentation, prior to the deadline.
- **T07:** Submitted before the stated deadlines.

Mandatory Requirements (Authentication & Profile)

- **MR22 & MR25 (Registration):** Users can register via the `/api/auth/register` endpoint. Passwords have complexity checks. If an email is already registered, an Angular interceptor catches the 409 Conflict error and displays a readable error message to the user (MR25).
- **MR23 & MR27 (Login):** Implemented via the login screen. It exchanges credentials for a JWT. Incorrect credentials result in an error message displayed directly on the form (MR27).
- **MR24 (Logout):** The navigation menu contains a Logout button that clears the JWT from `localStorage` and invalidates the session via the backend.
- **MR26 (Profile):** Accessible via the router, the Profile page displays the user's username, email, and activity status.
- **MR28 (Password Change):** The profile section includes a "Change Password" form. It verifies the old password and enforces complexity on the new one before updating.
- **MR29 (Account Deletion):** Users can permanently delete their accounts. This triggers a cascading delete of their user profile and associated data from the database.

Mandatory Requirements (Train Data & Table)

- **MR30 (Train Data Table):** The main Data View displays a paginated Angular table showing only the ID, Station, and Planned Departure to prevent clutter.
- **MR31 (Detailed Train Data):** Each row in the table has a "Details" button. Clicking this opens a detailed view (fetching from `/api/trains/{id}`) showing all properties of that specific train ride.
- **MR32 (Filtering) & SR12 (Recent Searches):** A search bar is bound to the table. Users can search by ID, Station, or Planned Departure. As per **SR12**, the app stores and displays the last 5 queries in a dropdown below the search bar for quick repeating.
- **MR39 (Sorting):** The user can click the "Planned Departure" column header to toggle ascending/descending sorts.

Mandatory Requirements (Data Uploads & Downloads)

- **MR33 (Data Upload) & SR7 (Drag and Drop):** Users can navigate to the Upload page to upload the provided sample `.csv` files. As per **SR7**, users can either click to select a file or **drag and drop** it directly into the designated drop zone area.
- **MR34 (Format Errors):** The backend actively validates the CSV headers. If the user uploads a malformed file, the backend rejects it, and the frontend displays a clear error alert explaining that the columns do not match the template.
- **MR35 (Download CSV) & SR11 (Export History):** A "Download Filtered Data" button exists on the table view. It downloads the currently visualized/filtered dataset as a new CSV. As per **SR11**, every time a user exports data, a timestamped record is saved and displayed in their Profile's Export History section.
- **MR40 (Delete Uploaded Data):** In the profile/upload management area, users can see their past uploads and click "Delete" to remove them from the system database.

Mandatory Requirements (Analytics & Background Tasks)

- **MR36 (Cancellation Odds):** On the Analytics page, users can select a Source and Destination station. The backend initiates a long-running calculation to determine the cancellation probability.
- **MR37 (Average Delay):** Users can select a “Bundesland” (Federal State). The system calculates the average delay time for all trains occurring in that region.
- **MR38 (Calculation Progress Status):** Because MR36 and MR37 are long-running tasks, the FastAPI backend returns a `task_id`. The Angular frontend uses RxJS `timer()` to poll the `/api/analytics/status/{task_id}` endpoint every 500ms, mapping the `progress_percentage` to an **Angular Material Progress Bar** (`<mat-progress-bar>`) so the user can visually track the calculation in real-time.

Mandatory Requirements (UX & Safety)

- **MR41 (Router Navigation):** The application relies heavily on `RouterModule` to facilitate Single Page Application (SPA) navigation across all features.
- **MR42 (Confirmation Dialogs):** Irreversible actions—specifically deleting the user account (MR29) and deleting uploaded data (MR40)—are protected by an **Angular Material Dialog** (`MatDialog`). The user is explicitly asked “Are you sure?” and must confirm before the deletion executes.

Should Requirements (UI/UX Extensions)

- **SR8 (Theme Switching):** A toggle in the user profile/navbar allows swapping between Light Mode and Dark Mode. This dynamically updates CSS variables and Angular Material’s theme configurations to adjust the entire interface.
- **SR9 (Language Switching):** A toggle switches the UI language between English and German, relying on standard `i18n/ngx-translate` pipelines.
- **SR10 (Delay Reason Overview):** A dedicated section on the Analytics page fetches and visualizes the top 5 most common reasons for train delays based on the aggregated data.

Can Requirements (Gamification)

- **CR4 (Reward Points):** To incentivize participation, users automatically receive reward points in their account every time they successfully complete a valid CSV train data upload (MR33).
- **CR05 (Points Display):** The user’s current accumulated point balance is prominently displayed at the top of their User Profile page.
- **CR06 (Leaderboard):** A dedicated Leaderboard page queries the backend for the top users. It displays the Top 5 users globally who have gathered the most points from uploading data, fostering competition.

Technical Implementation Details

(For Grading Reference)

Angular Polling for Background Tasks (MR38) To prevent UI freezing during calculations, long-running metrics use a non-blocking asynchronous approach:

```
calculateAverageDelay(bundesland: string): Observable<any> {  
  return this.http.post<{task_id: string}>(`${apiUrl}/analytics/average-delay`, { bundesland })  
    .pipe(  
      switchMap(res =>  
        timer(0, 500).pipe( // Poll every 500ms  
          switchMap(() => this.http.get<any>(`${apiUrl}/analytics/status/${res.task_id}`)),  
          takeWhile(status => status.status !== 'completed' && status.status !== 'failed', true)  
        )  
      )  
    );  
}
```

Thank you for a great semester, Mr. Holger. I hope you enjoy reviewing the application!